CSF205:
Web Service Development

Week 3: Web Technologies and Web Services



Swansea University
Prifysgol Abertawe

Department of Computer Science
Applied Software Engineering

Learning Outcomes



Lecture

- Object Oriented distributed programming.
- Core web technologies and where to find them.
- Web services and why they work.
- Two approaches to building web services.

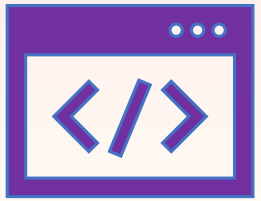


Lab

- Writing your first HTTP server
- Understanding how to interact with deployed web services.
- Analysing the differences between Web Service designs

PART 1

Object Oriented Distributed Programming and Web Technologies



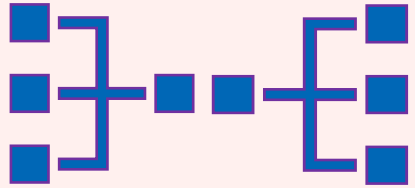
Object Oriented Approaches: Abstracting Sockets

Object Oriented paradigms gain traction in the late 1980s and 1990s.

Computer systems are becoming complex.

Distributed programming is becoming desirable.

Developers want to send objects over networks.



- Ideally, remote method / function / procedure calls would be transparent to developers and appear as if things were happening locally.
- Objects created in one language need to be interpreted and usable by other languages. This is known as marshalling and unmarshalling.

Doing this with sockets and packets can get verbose and complicated.



Object Oriented Approaches: In the beginning...

In the 1990s –2000s, there are a few different approaches to this:

Microsoft (Distributed) Component Object Model.

- Proxies, Stubs, Clients, Servers and Remote Procedure Calls.
- Built for Windows environments and networks

OMG Common Object Request-Broker Architecture

- Interface definition language (IDL) to specify what types and methods objects expose. IDL can be compiled to different languages
- Very complicated, but allows OS, language, network interoperability

Java Remote Method Invocation

- Objects accessible across different systems and Java Virtual Machines

Object Orientated Approaches: Java RMI

Designed to make communication between multiple Java programs, running in separate JVMs, as much like making a method call inside a single process as possible.

Uses a **client-side stub**:

- An object in the client's JVM to represent the server-side object.

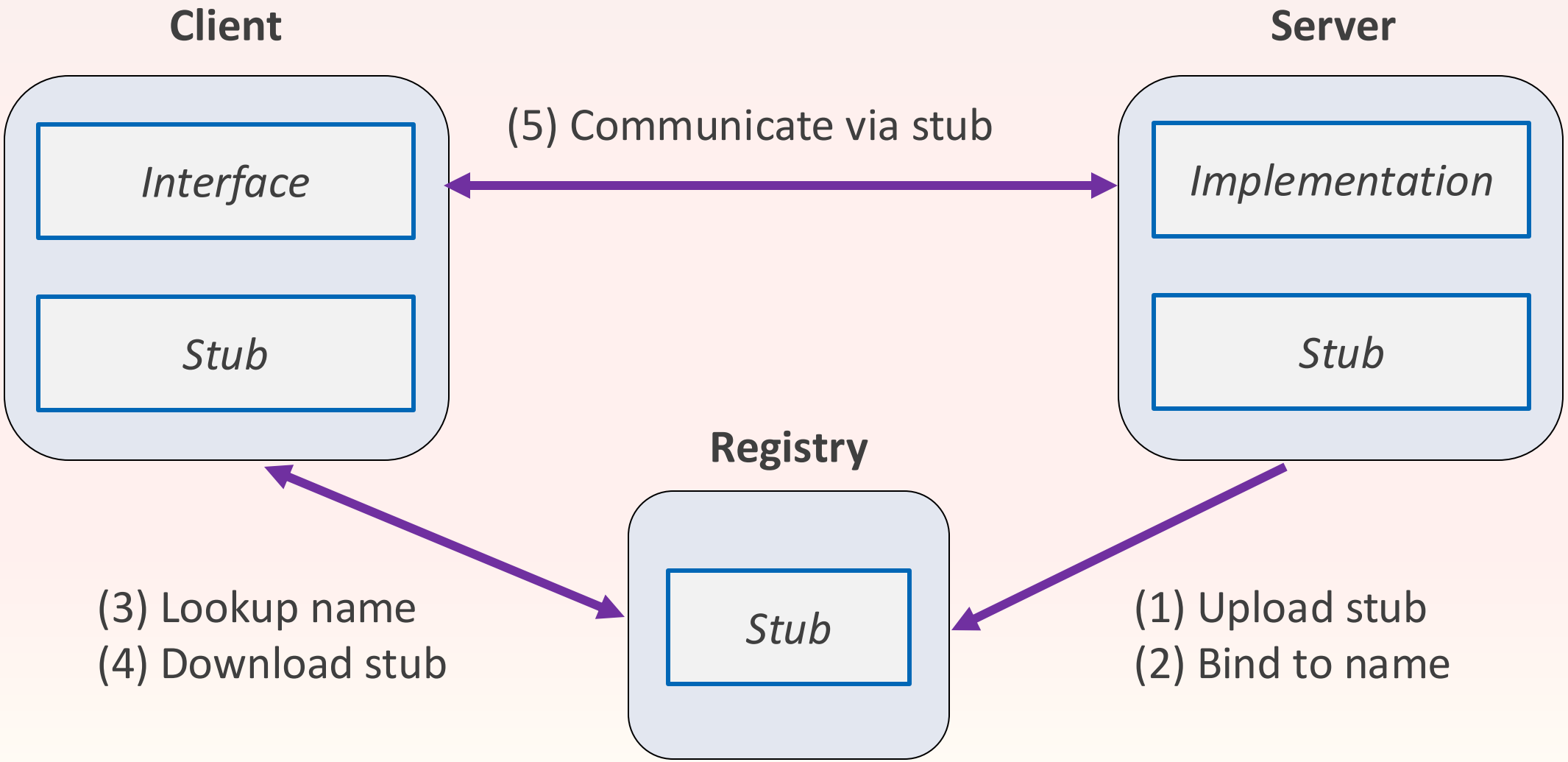
Uses a **server-side stub**:

- Object maintaining network connections, marshalling and unmarshalling data on server side.

Central naming service called rmiregistry:

- Functions like a phone book, providing the server associated with a naming service.

Object Orientated Approaches: Java RMI Stages



DEMO

Java RMI

Object Oriented Approaches: RMI Concerns

- Java RMI was useful – it is truly Object Oriened and works as expected.
- Servers could run any code sent by the client, and vice versa. But...
- Registry isn't necessary for things to work, and it isn't managed for you.
- It was essentially Java-only. Not great for platform independence.
- Both client and server JVMs must be compatible.
- RMI allows you to send runnable code, not just messages, across machines.



Concept Check: Remote Code Execution

What problems might occur if we can send arbitrary runnable code over the internet?



Object Oriented Approaches: (In)secure RMI

- Arbitrary Remote Code Execution is quite literally a security nightmare.
- Sending runnable code over the internet is dangerous!
- There were security measures in place, but many people didn't use them.
- These things also tend to get blocked by firewalls.
- Which RMI was aware of... and would try to disguise itself as web traffic.

- Generally, a very bad idea outside of private, closed networks.



Object Oriented Approaches: What about us?

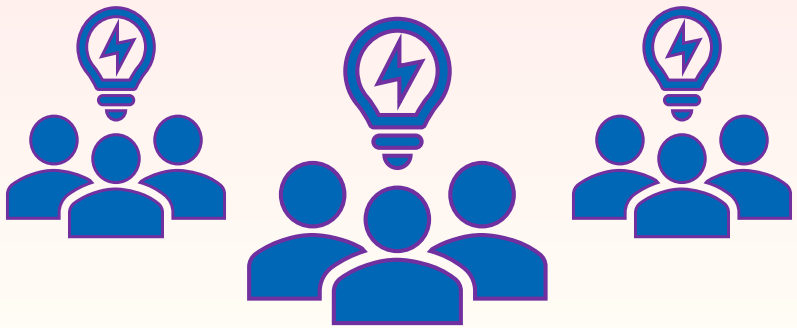
We no longer do Object Oriented distributed programming.

But approaches have been designed to look like we still do.

Programmers need to be aware:

- What you see is not always what you get
- Expectations may clash with reality

Over the coming weeks we see how we can send data, not code, over the internet in a way that mostly aligns with a familiar style of programming.



The Web: Origins of the World Wide Web

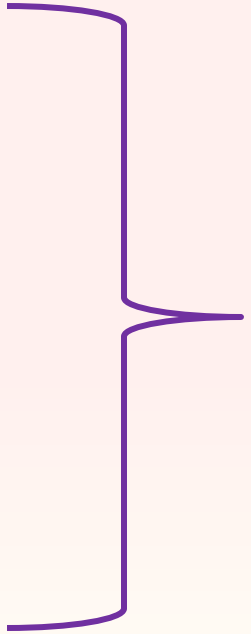
Invented by Tim Berners-Lee at CERN (1989) to accelerate structured information sharing.

Resulted in three core web technologies:

HTML (Hyper-Text Markup Language)
- How to correctly show data.

URL (Uniform Resource Locator)
- How to find that data.

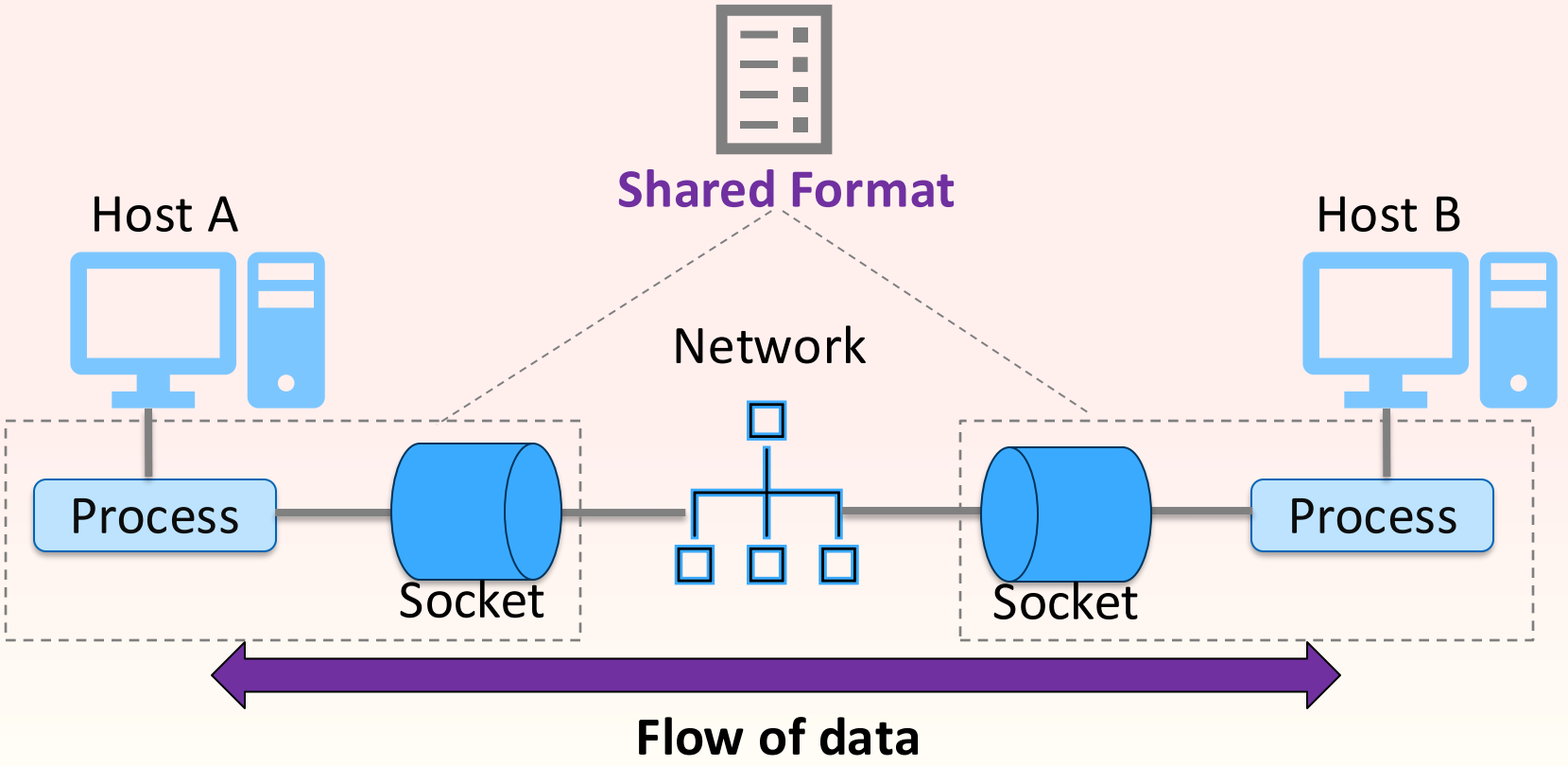
HTTP (Hyper-Text Transfer Protocol)
- How to move data from one machine to another.



We'll look at each of these in turn.

The Web: Motivating Markup Languages

- Standardised description of structure and meaning of arbitrary data.
- Described using (mostly) tags and elements.
- Interpretation becomes platform dependent.
- Separates content and presentation.

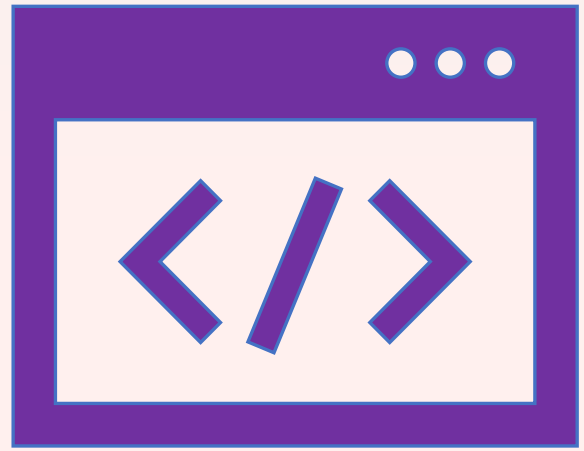


The Web: HTML

Markup system composed of:

- Elements
- Attributes
- Tags

which represent specific content.



- HTML tags provide semantic meaning
- Do not specify or impose logic or describe behaviour.
- Defines *what* is meant to be parsed by the browser.
- Does not concern *how* content is presented.



The Web: HTML Structure

Tag	Purpose
`<!DOCTYPE>`	• Defines HTML version used in the document • Defaults to HTML 5
`<html>`	• Container for elements rendered by browser
`<head>`	• Contains HTML document metadata • Declares imports from external stylesheets and scripts. • Does not appear in the main browser window
`<body>`	• Contains visible or audible content of the page



The Web: HTML Formatting

Component	Purpose
`<h1>,<h2>,…,<h6>`	Headers for structuring text, from largest to smallest.
`<p>`	Contain and separate paragraphs
`<em>, <strong>, <i>, <b>`	Highlight text with boldface or italics
` `	Force whitespace
` `	Newline
`<pre>`	Renders text with a fixed font, preserving whitespace and newlines
`<code>`	Fragment of computer code
`<figure>`	Single container for other ‘flow’ content
`<blockquote>`	Highlights quotations that can be referenced.
`<mark>`	Tags content of relevance
`<ul>, <ol>, <li>`	Create ordered and unordered lists with list items
`<dl>, <dt>, <dd>`	Create list of definitions with a formatted description

The Web: Uniform Resource Identifiers / Locators

Okay, so we can:

- *Present* data in a uniform way using markup languages.
- *Identify* machines and processes over networks using TCP/IP.

What about individual resources (documents, images, videos)?

- Berners-Lee proposes the Uniform Resource Identifier (URI).
- URIs are a superset of the Uniform Resource Locator (URL)
- URIs identify resources, URLs are how you locate them.
- All URLs are also URIs - they are also structurally identical...

The Web: URI / URL Format

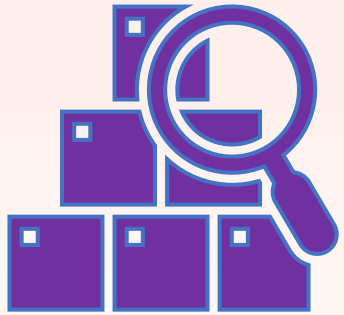
Scheme Authority Path

https://example.com:8080/store/login.html

What resource am I trying to access?

Which machine and process is hosting it?

Which **protocol** does it use to communicate?



The Web: HTTP Methods

They provide us the actions needed to interact with static resources.

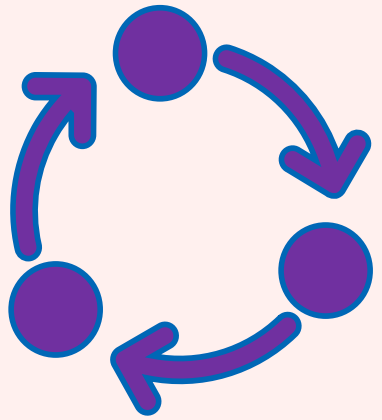
POST: Creates and pushes data to the recipient

GET: Reads data from the request recipient.

PUT: Updates some data held by the recipient.

DELETE: Deletes a resource held by the recipient.

HEAD: Requests checks for a resource without retrieving it.



There are others, but we won't be using them.

The Web: HTTP Responses

Each HTTP request is met with an HTTP response.

The nature of that response is "summarised" by a 3-digit response code.

There are many different codes for different purposes.

Codes fellow a pattern and are classified according to the first digit.

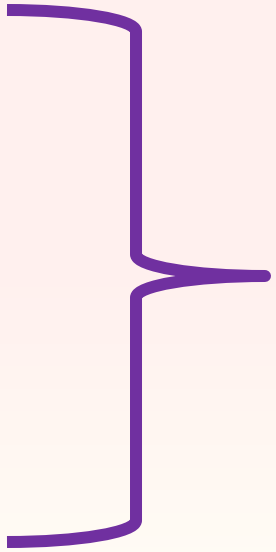
1 - (Information)

2 - (Success)

3 - (Redirection)

4 - (Client Fault)

5 - (Server Fault)



Very useful for conveying information in terms of client-server interaction.

The Web: HTTP Headers

Each HTTP requests/responses includes a section called the header.

This contains a lot of protocol and application specific data.

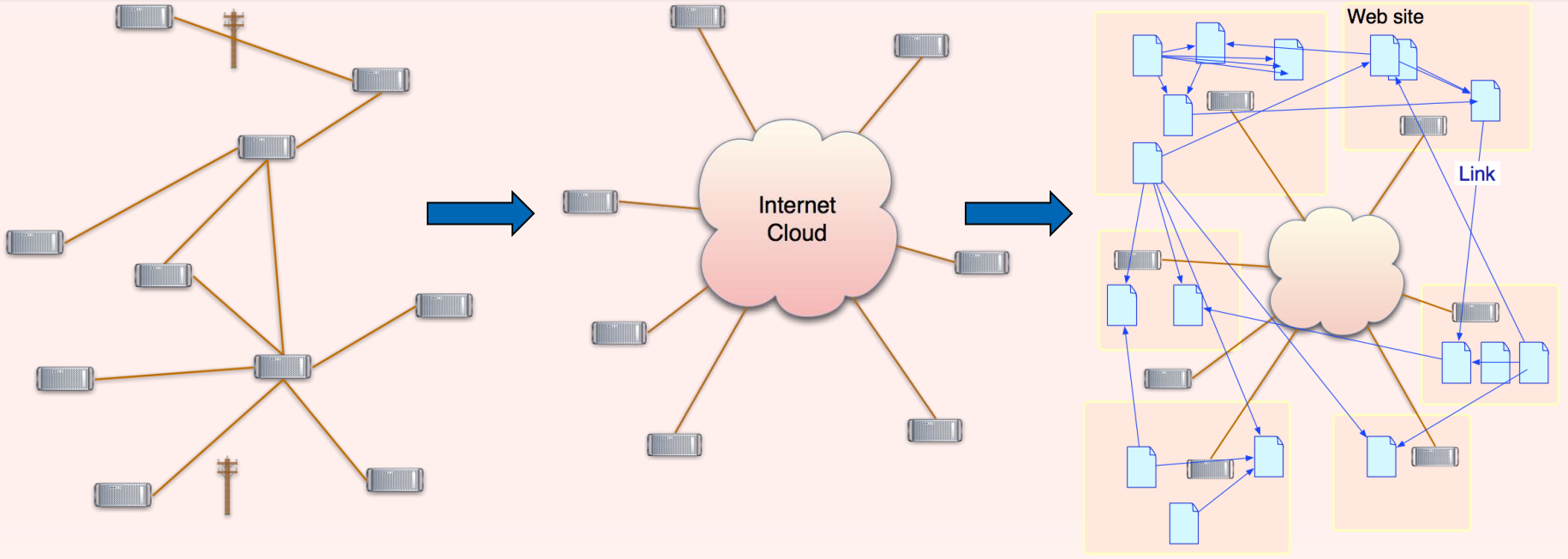
That data uses a "Name : Value" format.

```
Host: swansea.ac.uk
Accept: text/html,application/xml,application/json
Date: Wed, 9 Oct 2024 14:34:08 GMT
```

- Many of these are optional.
- Some describe the data the underlying HTTP body is carrying.
- Not the only way to pass parameters between client and server

The Web: WWW and Distributed Computing

Wired Machines → Files between Machines → Interactive documents



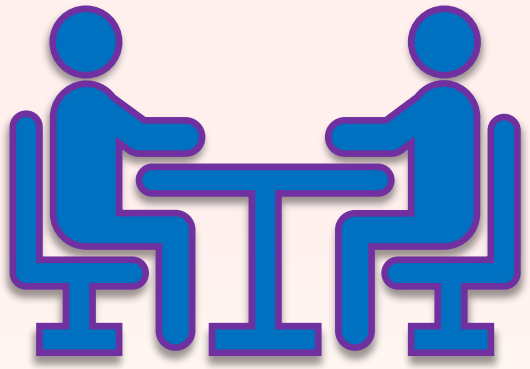
Courtesy: Tim Berners Lee

HTML empowered exchange of information.

But what about exchange of services?

Summary: Part 1

- Traditional approaches to distributed programming aimed to use Object Oriented paradigm across machines
- An early approach to this in Java was RMI
- This comes with baggage, mostly security problems.
- A way of addressing this is designing system interaction to rely on existing web technologies, like HTTP



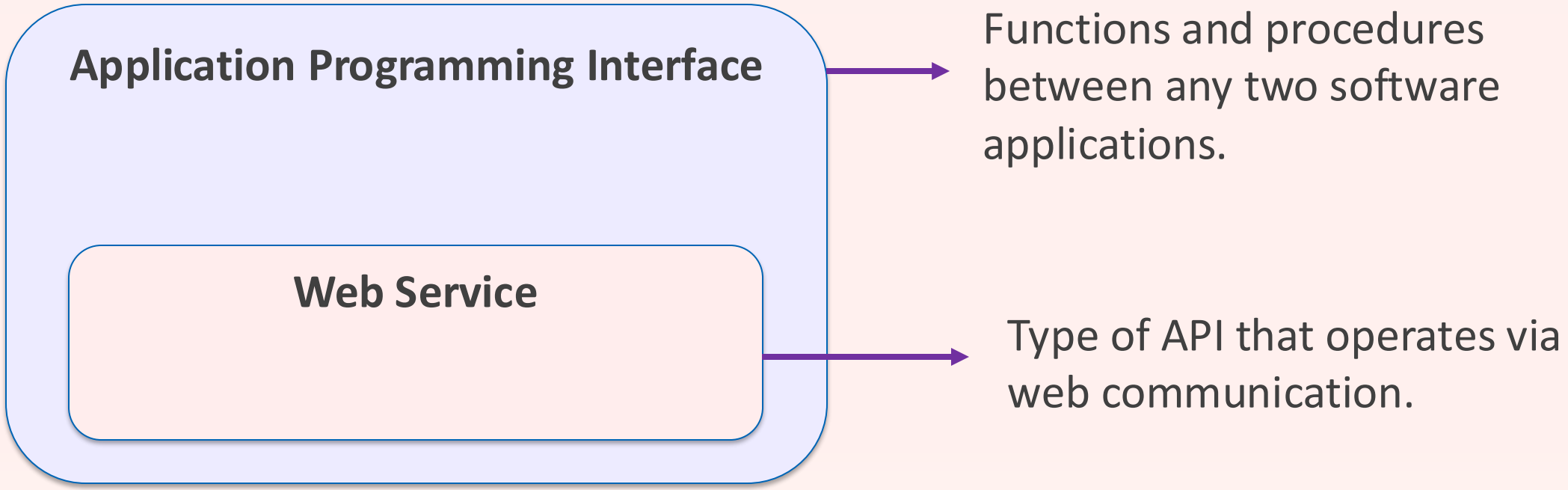
Next lecture: Web Services

PART 2

Web Services, SOAP and REST



Web Services: Web Service or API?



Web Services: Desirable Properties

Open Infrastructure:

- Promote interoperability among services.
- Use industry-standard, vendor-independent protocols (HTTP, XML)
- Piggyback on existing networking, data formatting, security etc.

Language Transparency:

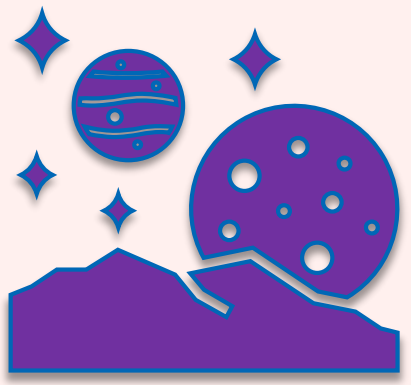
- Clients and services can interoperate irrespective of their programming language or underlying OS.

Modular Design:

- Meant to be modular in design so that new services can be generated through the integration and layering of existing services.

Web Services: Landscape

- HTTP and URLs allow clients to retrieve HTML pages from servers.
- Unlike the RMI/CORBA/DCOM model, calling specific functions was not not possible.
- Exchange of data was possible to an extent with XML (eXtensible Markup Language).
- Running specialised services was still limited.



In come web services:

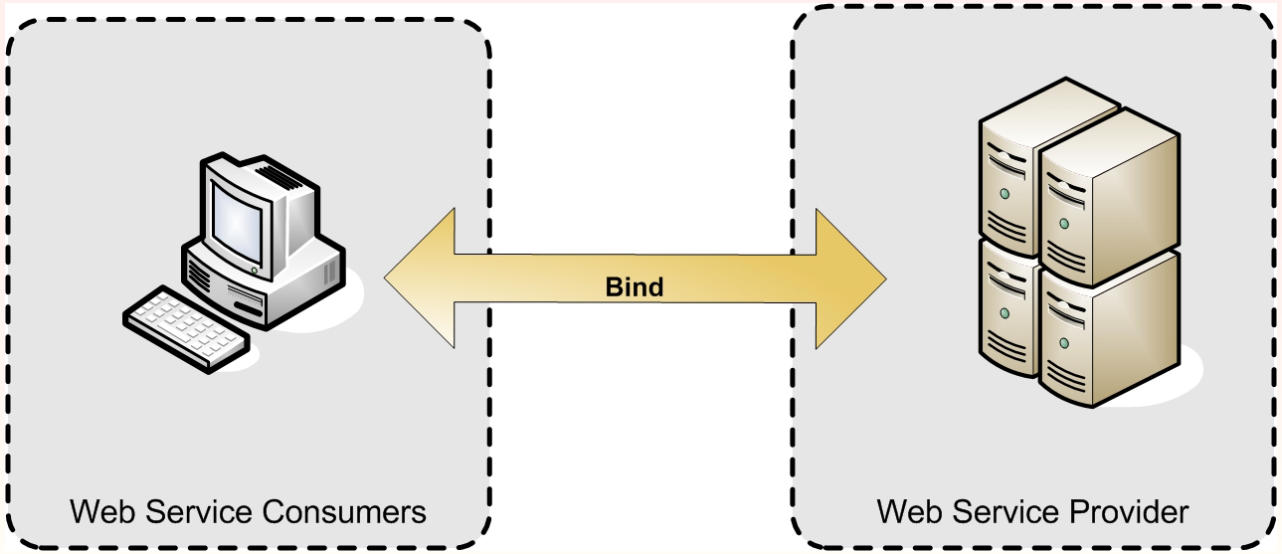
- Application-specific clients can interact with a service via a specialised interface using existing network architecture and web technologies.

Web Services: The (General) Architecture

The simplest Web service system has two participants:

- A service producer (provider)
- A service consumer (requester)

The provider presents the interface and implementation of the service, and the requester uses the Web service.



Web Services: The Basic Requirements

Message passing framework:

- Handling errors and faults in application level to ensure fault tolerance.

Intermediaries:

- Augmenting message with metadata and ways to monitor, route, or transform messages.

Common interface definition:

- Common way to expose a set of operations/services.

Easy location of services:

- Organising the web services via registries to ensure that they are found easily by users.

Web Services: The Service Orientated Architecture (SOA)

Designing a software system that

- Is composed of largely independent services.
- Uses standard protocols to communicate data.
- Publishes interfaces to support interoperability
- Can be distributed over networks.
- Aims to make its services extensible.

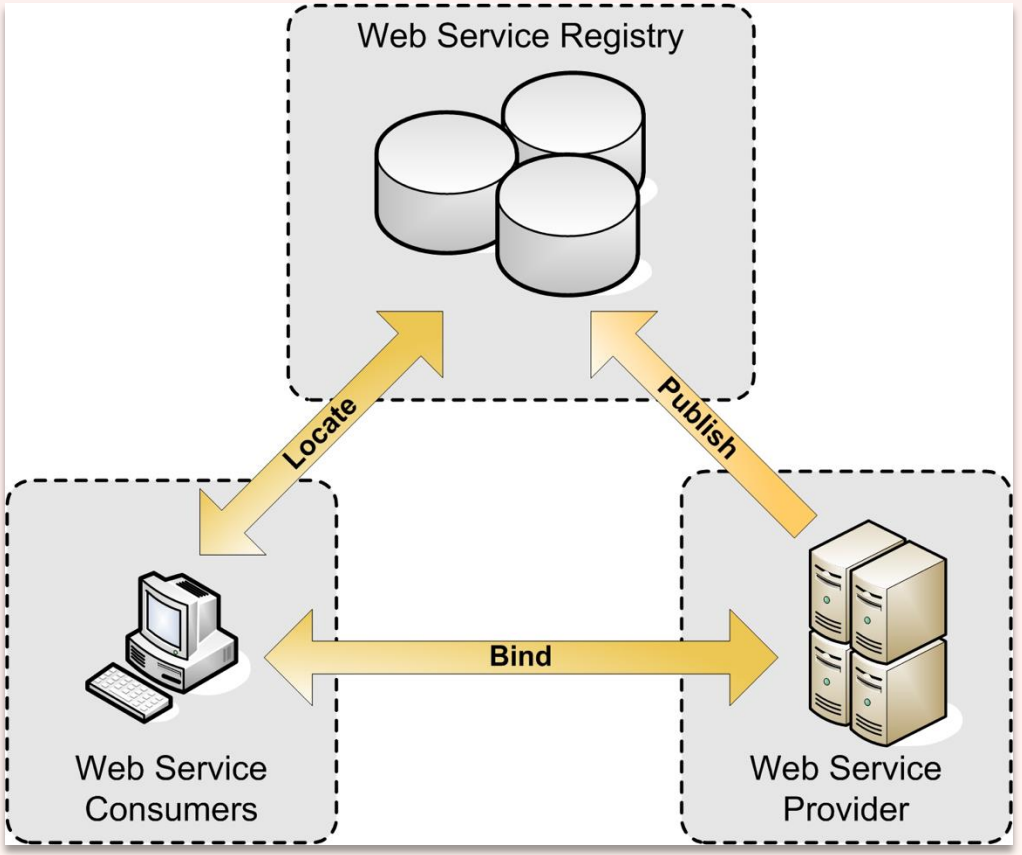
A web service is one of the most common instances of a SOA implementation.

Web Services: The Service Orientated Architecture

A more sophisticated system:

- A registry, acts as a broker for Web services.
- A provider, can publish services to the registry
- A consumer, can then discover services in

So how did developers agree to design this?

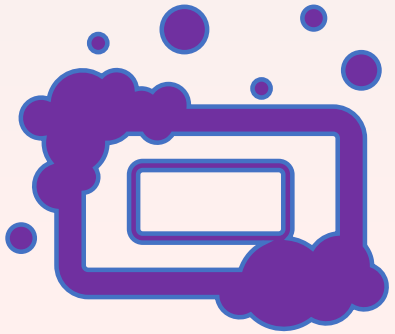


Web Services: SOAP

Originally stood for Simple Object Access Protocol.

Later, Service Orientated Architecture Protocol.

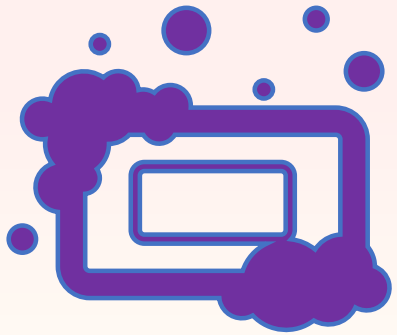
Now it just means SOAP...



- An early attempt at meeting this specification, starting at Microsoft, in the mid-late 1990s.
- Uses XML messages, or envelopes, to encode method calls and pass parameters.
- Uses HTTP (and other transport protocols) to exchange messages.
- Can pass information describing how data should be interpreted

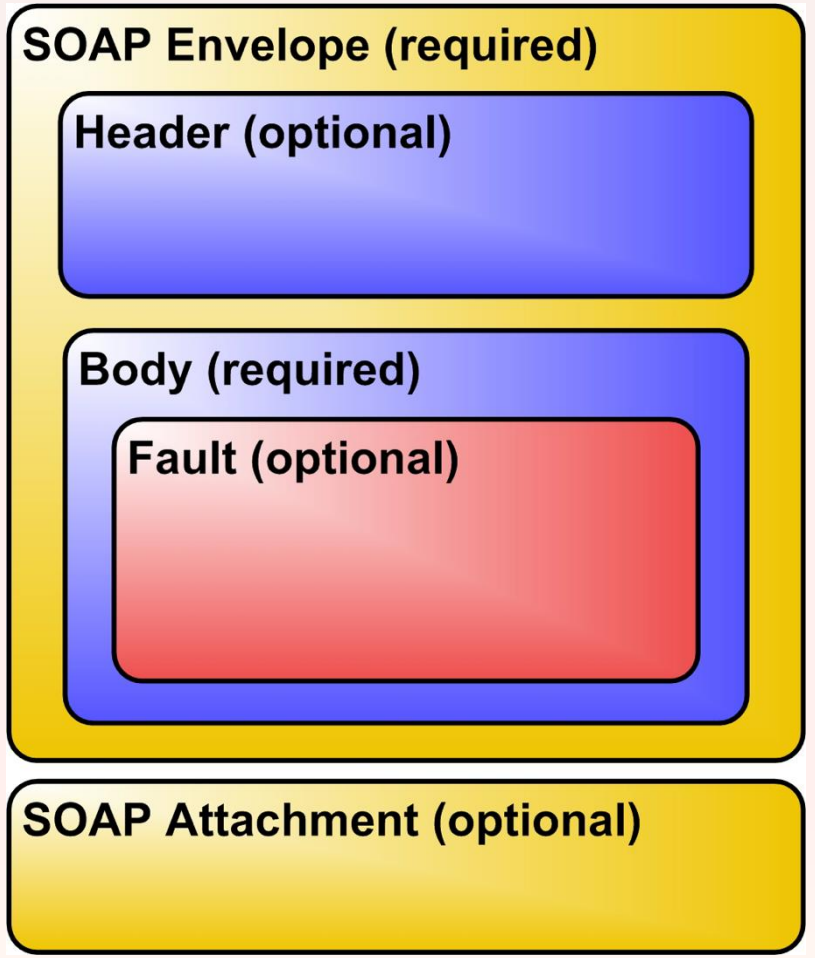
Web Services: The Good SOAP

- XML became well-established, meaning tools for parsing and generating.
- XML messages also appeared to be 'legitimate' traffic, reducing the security concerns (but not completely preventing them)
- No executable code is being passed, just information for RPCs to work.
- Is truly platform independent and means client-server applications could be loosely coupled.



Web Services: The SOAP Message

Element	Description	Required?
Envelope	Top-level XML element	Yes
Header	Metadata about SOAP message	No
Body	Data sent for receiver to process	Yes
Fault	Any Exceptions and errors	No



DEMO

Postman and SOAP

Web Services: SOAP Request

```
POST /UserService HTTP/1.1
Host: www.example.com
Content-Type: text/xml; charset=utf-8
Content-Length: length
SOAPAction: "getUserDetails"
```

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:usr="http://www.example.com/userservice">

   <soapenv:Body>
      <usr:getUserDetails>
         <usr:userId>12345</usr:userId>
      </usr:getUserDetails>
   </soapenv:Body>
</soapenv:Envelope>
```

Web Services: SOAP Response

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: length
```

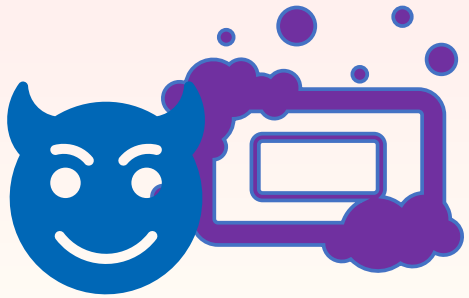
```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:usr="http://www.example.com/userservice">

   <soapenv:Body>
      <usr:getUserDetailsResponse>
         <usr:user>
            <usr:userId>12345</usr:userId>
            <usr:firstName>John</usr:firstName>
            <usr:lastName>Doe</usr:lastName>
            <usr:email>john.doe@example.com</usr:email>
         </usr:user>
      </usr:getUserDetailsResponse>
   </soapenv:Body>
</soapenv:Envelope>
```

Web Services: The Not So Good SOAP

SOAP works fine but:

- XML messages are verbose.
- Not the most pleasant things to look at.
- Message sizes mean they're not computationally "cheap" to process.
- The way data is passed and the way methods are called are separate.
- Operations, data types, parameters need a client-server contract

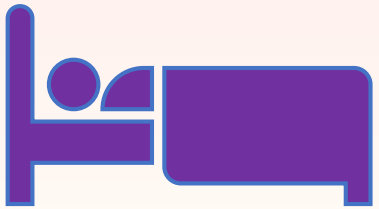


Web Services: The REST Alternative

Representational State Transfer (REST)

Focuses on using HTTP, URIs and a simpler markup language (JSON):

- No longer interact with services, but with resources.
- URIs identify the endpoints for those resources (and parameter passing)
- HTTP methods describe the operations you can perform
- HTTP codes and headers can carry additional information
- JSON much simpler to write, read and less verbose.
- Also supports XML



Web Services: Resources

Consider the URI:

`https://www.mywebsite.co.uk/stuff/morestuff/thedeepeststuff`

Each aspect of the URI indicates a resource held by the server.

REST allows URIs to represent a resource and associated parameters.

`https://www.mywebsite.co.uk/resource/parameter1/parameter2`

The HTTP method used with that URI then defines what operation will be performed on the server, using those parameters, for that specific resource.

Web Services: REST Request

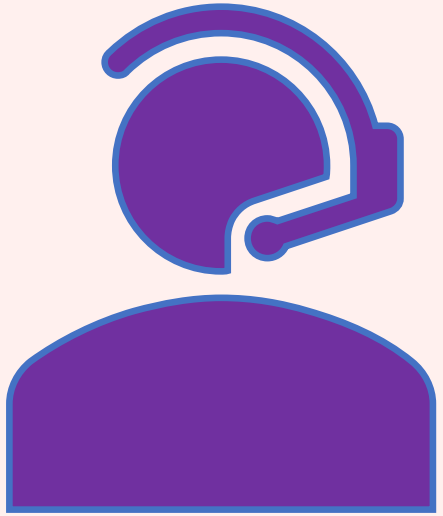
```
GET /users/12345 HTTP/1.1
Host: www.example.com
Accept: application/json
```



Yes, that's quite literally it.

Web Services: REST Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: length
{
  "userId": 12345,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com"
}
```

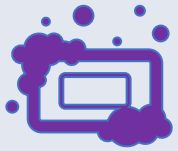


DEMO

Postman and REST

Web Services: SOAP and REST

SOAP



A **protocol** following RPC architectural style.

Must represent messages in XML.

Relies heavily on HTTP for transferring the XML payload

REST



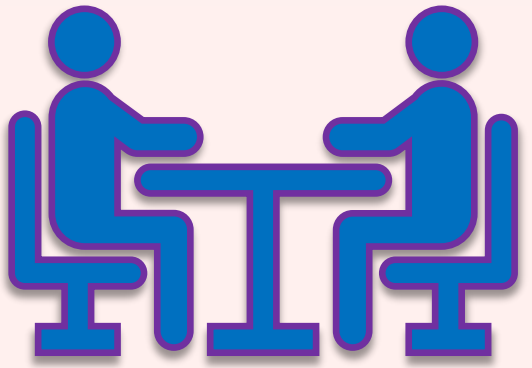
An **architectural style**

Supports multiple data representation languages.

Uses HTTP protocol and URI for methods and scoping information.

Summary: Part 2

- Transitions from traditional distributed programming meant services needed to exchange data, not code
- Growing demand for standardisation, interoperability and platform independence
- SOAP was an early solution to the above, using XML and web transport to send SOAP Messages.
- REST is a ‘stripped down’ alternative, with some trade-offs on more emphasis on HTTP.
- These trade-offs will become apparent over the coming weeks



Lab: HTTP and Calling Web Services

Next lecture: REST Design